Artificial Intelligence

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 16 - PAO 2024 – II

Agenda

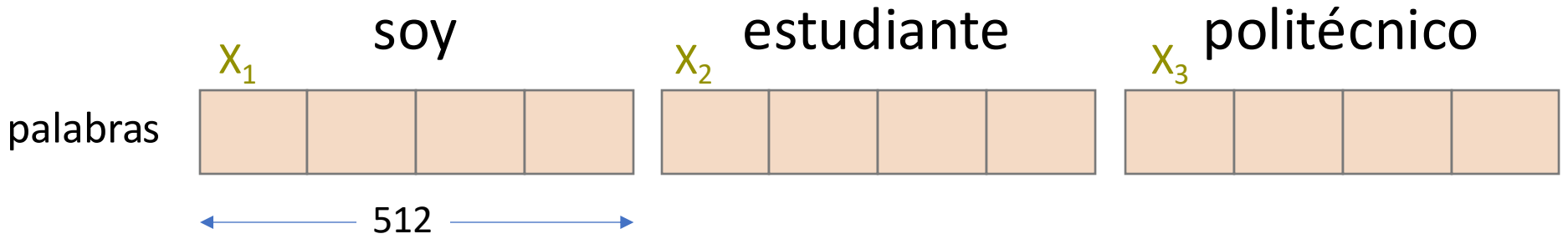
1. Redes neuronales recurrentes - RNN
    - 1.1 Embeddings
    - 1.2 Procesamiento de Lenguaje Natural
    - 1.3 Ejemplo RNN

Redes neuronales recurrentes

Modelado del lenguaje

- La base del modelamiento de lenguaje se denomina *n-gram*
- Un *n-gram* es un conjunto de *n* palabras consecutivas
- Por ejemplo, en la oración: "Estoy escribiendo un ...", tendríamos los siguientes tipo de *n-grams*:
  - unigrams: ”Estoy", "escribiendo", ”un”
  - bigrams: "Estoy escribiendo", "escribiendo un”, "un …”
  - trigramas: "Estoy escribiendo un”, "escribiendo un …”
  - 4-grams: "Estoy escribiendo un …”

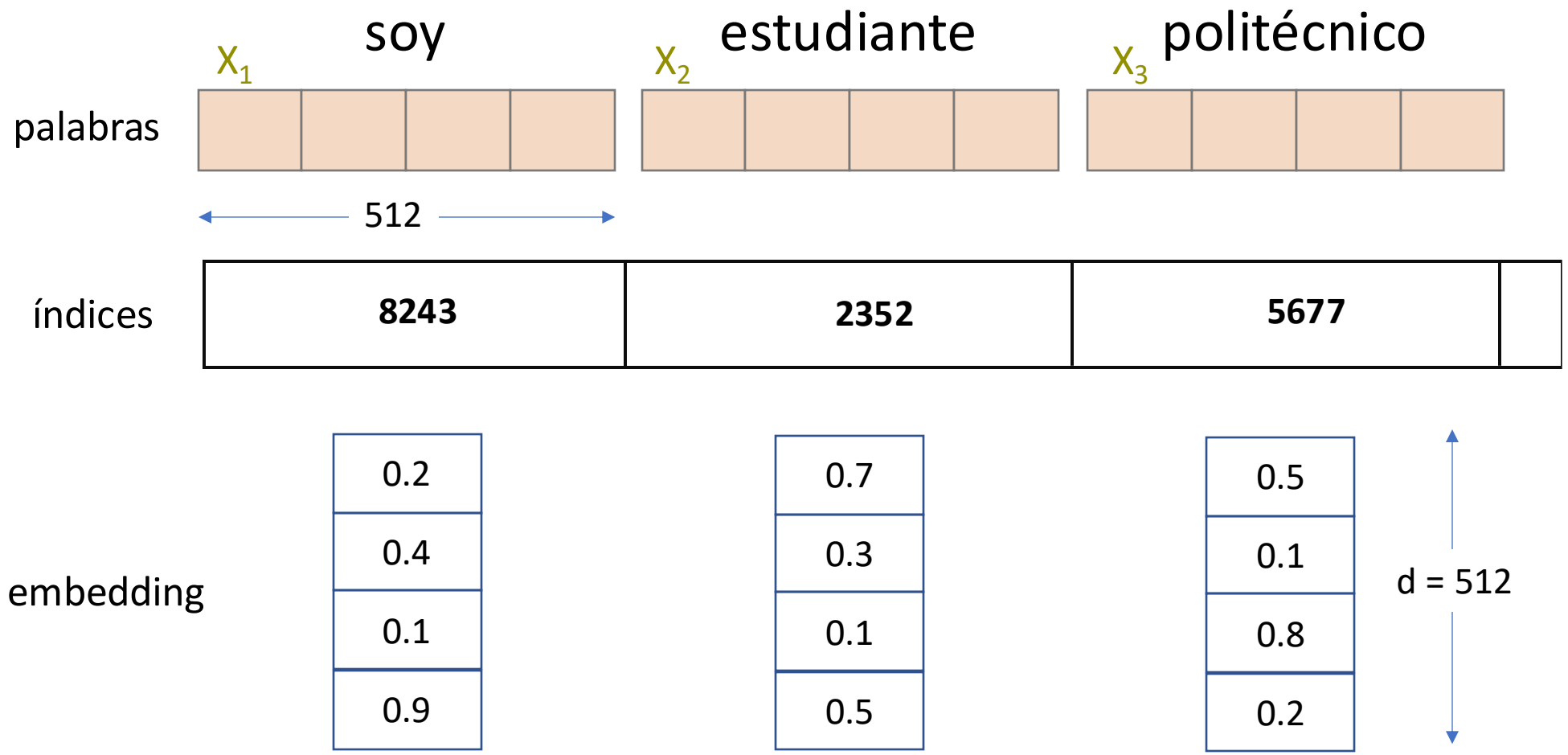
Redes neuronales recurrentes: Embeddings



Qué es un embedding?

- Es una representación numérica de texto, donde las palabras con igual significado tienen una representación similar
- Es un vector d-dimensional para cada número o índice de representación de cada palabra
- Si tenemos un diccionario de 10000 palabras, necesitamos 10000 índices
- Cada palabra tiene un índice único y cada índice tiene un vector embedding

Redes neuronales recurrentes: Embeddings



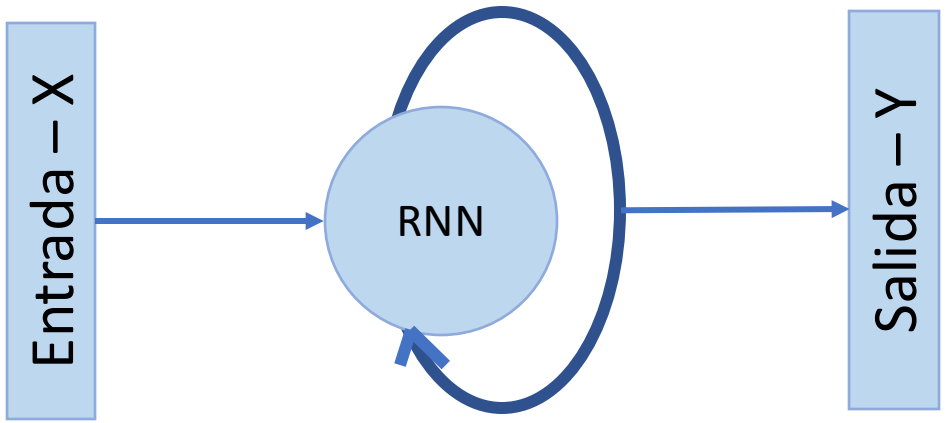
Redes neuronales recurrentes: Embeddings

Tipos de Embeddings:

1. Estáticos:
   - Son números únicos que representan las palabras. Estos pueden ser valores pre-entrenados de las palabras, que luego se podrían usar mientras entrenamos nuestros modelos.
     Por ejemplo: Glove (Latent Semantic Analysis-LSA) y word2vec (Continuous Bag-of-Words-CBOW, Skip-Gram) son embeddings estáticos. Están previamente entrenados.
     Los modelos independientes del contexto, como word2vec o GloVe, generan una representación de cada palabra en el vocabulario.
2. Contextual:
   - Cada palabra en los documentos tiene una representación independiente y continua (vector), que depende de su uso
   - Por ejemplo: ELMo, BERT son representaciones continuas y se pueden usar en tareas de aprendizaje posteriores.
   - Los modelos contextuales generan una representación de cada palabra basado en las otras palabras de la oración.

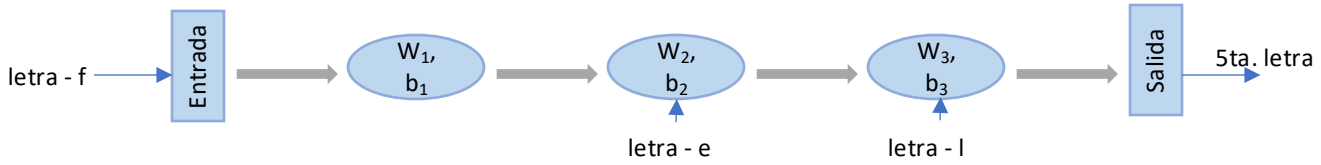
Redes neuronales recurrentes

- Veamos un ejemplo: Supongamos una RNN, con la que queremos predecir caracteres
- Supongamos que tenemos la cadena de caracteres “feliz”; y, luego de ingresar las primeras 4 letras: “f, e, l, i”, deseamos que la RNN prediga la siguiente letra: “z”
- En este ejemplo el vocabulario tiene 5 letras: {f, e, l, i, z}, este vocabulario se denomina “Corpus”, en la práctica, este puede llegar a tener miles de millones de palabras (la base completa de Wikipedia, o toda la base de google)
- Veamos cómo esta estructura se utiliza para predecir la siguiente letra en la palabra “feliz”

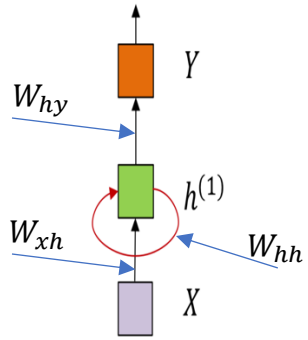


Redes neuronales recurrentes

- Ingresamos las letras en secuencia:
  - Ingresamos primero la letra ”f", la cual no tiene nada que la preceda, por lo tanto el estado anterior es 0
  - Luego de calcular el estado de la red en este time step, h_t, ingresamos la letra ”e”
  - En el momento que suministramos la letra ”e", se aplica la fórmula de recurrencia a esa letra, y al estado anterior de la capa oculta, en este caso la letra ”f”
  - Este es el proceso conocido como pasos de tiempo o time steps
  - Si en el momento t, la entrada es ”e", en el tiempo $t - 1$, la entrada fue ”f"
  - Para obtener el nuevo estado, la fórmula de recurrencia se aplica tanto a “e” como a “f”, así:



Redes neuronales recurrentes



- $h_t = f_1(h_{t-1}, X_t)$
- h_t es el nuevo estado, h_{t-1} es el estado anterior, y X_t es la entrada actual
- Supongamos que la función de activación f_1 es *tanh*; que los pesos en la neurona recurrente es W_{hh}; y, que los pesos en la neurona de entrada es W_{xh}
- Entonces la ecuación que determina el estado h_t en el momento t es:

$$h_t = tanh(W_{xh}X_t + (W_{hh}h_{t-1} + bias))$$

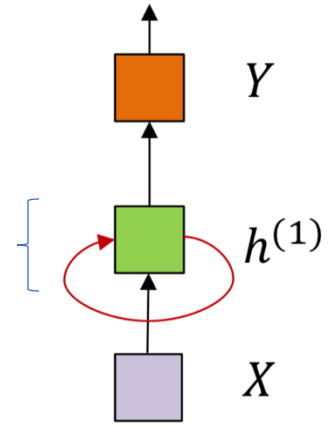
Redes neuronales recurrentes

- Veamos el paso “forward” de este ejemplo:

f	e	l	i
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

- Supongamos que el embedding de las entradas están en formato: one-hot-encoded
- El vocabulario completo es: {f, e, l, i, z}

Redes neuronales recurrentes



- Supongamos una capa oculta con 3 neuronas de ancho
- Entonces necesitamos una matriz W_{xh} de dimensión 3 (neuronas) x 4 (# de time-steps)

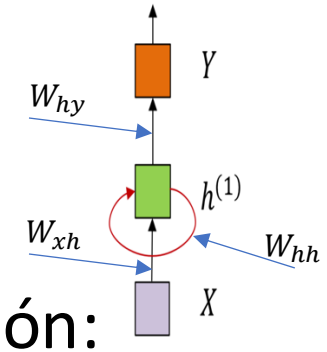
W_{xh}			
0.287027	0.84606	0.572392	0.486813
0.902874	0.871522	0.691079	0.18998
0.537524	0.09224	0.558159	0.491528

W_{hy}		
0.37168	0.974829459	0.830034886
0.39141	0.282585823	0.659835709
0.64985	0.09821557	0.334287084
0.91266	0.32581642	0.144630018

W_{hh}
0.427043

Bias
0.56700

Redes neuronales recurrentes



- Para el cálculo de la salida de la capa oculta, usamos la ecuación:

$$h_0 = tanh(W_{xh}X_0 + (W_{hh}h_{0-1} + bias))$$

- El cálculo de la entrada a la capa oculta, correspondiente a la letra "f", en el paso 0, es:

W_{xh}			
0.287027	0.84606	0.572392	0.486813
0.902874	0.871522	0.691079	0.18998
0.537524	0.09224	0.558159	0.491528

$\times$

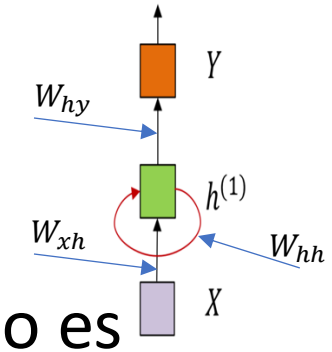
X_0
1
0
0
0
f

T

$=$

$W_{xh}X_0$
0.287027359
0.902874425
0.537523791

Redes neuronales recurrentes



- Para la letra “f”, como no tenemos un estado previo, el estado es [0, 0, 0], entonces el estado actual, en el paso 0, es:

$$h_0 = tanh((W_{hh}h_{0-1} + bias) + W_{xh}X_0)$$

- En el cálculo $W_{hh}h_{t-1}$, no olvidar el bias: $W_{hh}h_{t-1} + bias$

W_{hh}
0.427043

$\times$

h_{0-1}
0
0
0

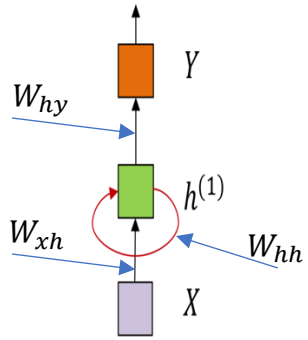
^{T} $+$

Bias
0.56700

$=$

$W_{hh}h_{0-1}$+bias
0.567001
0.567001
0.567001

Redes neuronales recurrentes

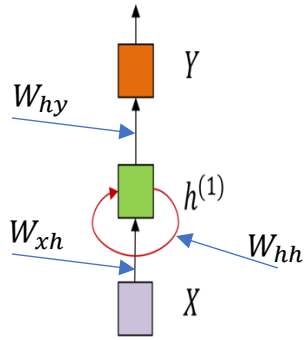


- Entonces: $h_0 = tanh(W_{xh}X_0 + (W_{hh}h_{0-1} + bias))$, es:

$$h_0 = \tanh\left\{\begin{bmatrix}0.287027359\\0.902874425\\0.537523791\end{bmatrix} + \begin{bmatrix}0.567001\\0.567001\\0.567001\end{bmatrix}\right\}$$

$$h_0 = \begin{bmatrix}0.69316804\\0.89955366\\0.80211840\end{bmatrix}$$

Redes neuronales recurrentes

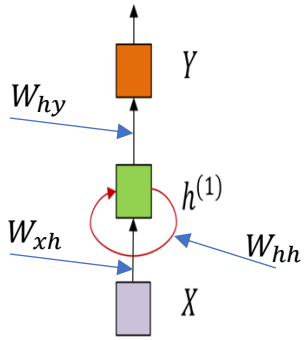


- Entonces el calculo de $h_1 = tanh(W_{hh}h_0 + W_{xh}X_1)$, es:

$$h_1 = \tanh\left\{\begin{bmatrix}0.863013\\0.951149\\0.90954\end{bmatrix} + \begin{bmatrix}0.846060\\0.871522\\0.092240\end{bmatrix}\right\}$$

$$h_1 = \begin{bmatrix}0.93653372\\0.94910403\\0.76234057\end{bmatrix}$$

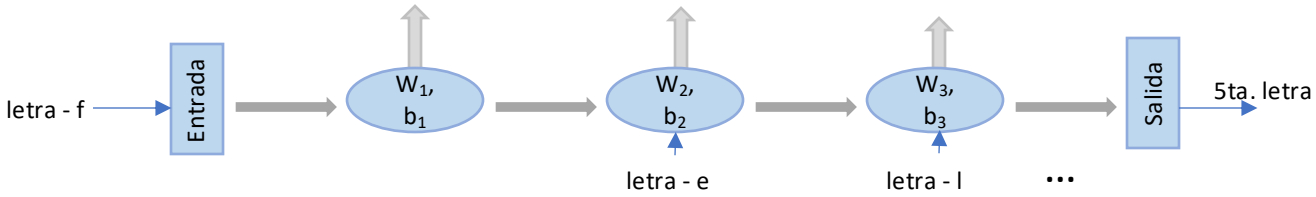
Redes neuronales recurrentes



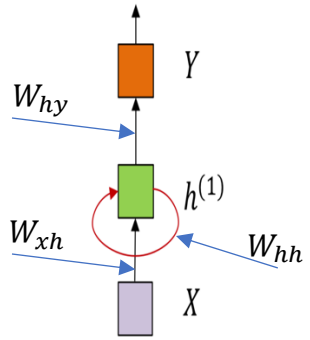
- Entonces el calculo de $h_1 = tanh(W_{hh}h_0 + W_{xh}X_1)$, es:

$$h_1 = \tanh\left\{\begin{bmatrix}0.863013\\0.951149\\0.90954\end{bmatrix} + \begin{bmatrix}0.846060\\0.871522\\0.092240\end{bmatrix}\right\}$$

$$h_1 = \begin{bmatrix}0.93653372\\0.94910403\\0.76234057\end{bmatrix}$$



Redes neuronales recurrentes



- Finalmente, calculamos la probabilidad de las letras en el vocabulario, por ejemplo aplicando la función de umbral softmax
- La letra en particular que el sistema recomendaría, en nuestro ejemplo, después de “e”, sería la letra que obtenga la mayor probabilidad, luego de aplicar softmax
- En nuestro ejemplo:

Y_t = Softmax

Y_1
1.90607732
1.13779113
0.95666016
1.27422602

=

Probabilidades
0.419748
0.194682
0.162429
0.223141

Redes neuronales recurrentes

- Estas probabilidades lo que nos dicen es que la letra después de "e" debe ser “f”, que tiene la probabilidad más alta
- La predicción no es la mejor, puesto que apenas hemos empezado a entrenar la red
- Acabamos de mostrarle dos letras. Así que prácticamente no ha aprendido nada todavía
- Una vez que concluya la última letra “i”, cómo se actualizan los pesos?
- Como en los modelos anteriores estas redes, “aprenden” también a través de backpropagation, que en RNN se lo llama “Back propagation through time” (BPTT)